



CS2009

DESIGN AND ANALYSIS OF ALGORITHMS

Dr. Hashim Yasin

**National University of Computer
and Emerging Sciences,
Faisalabad, Pakistan.**

Course Details

2

- **Credit Hours: 3+0**
- The course introduces the **basic & advanced techniques in analysis of algorithm.**
- **It helps to design and analysis of high-performance and scalable algorithms** as well as their engineering applications.

Course Details

3

- The Course Objectives are:
 - ▣ To develop skills needed for designing the correct and efficient algorithms,
 - ▣ To inculcate skills to understand mathematical notations in algorithms and their simple mathematical proofs,
 - ▣ To understand basic data structures and to use them in algorithms,
 - ▣ To gain familiarity with a number of classical problems that occur frequently in real-world.

Tentative Marks Distribution

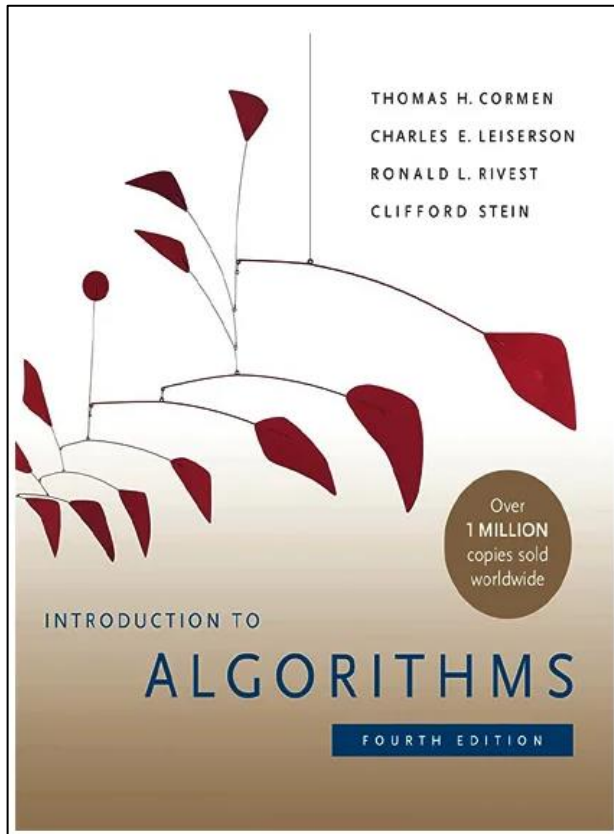
4

Item Name	Marks (%)
Quizzes	10-15
Assignments / Project	10-20
Sessional Exams	10-15 each
Final Exam	40-50

Grading Scheme = Absolute Grading

Recommended Books

5

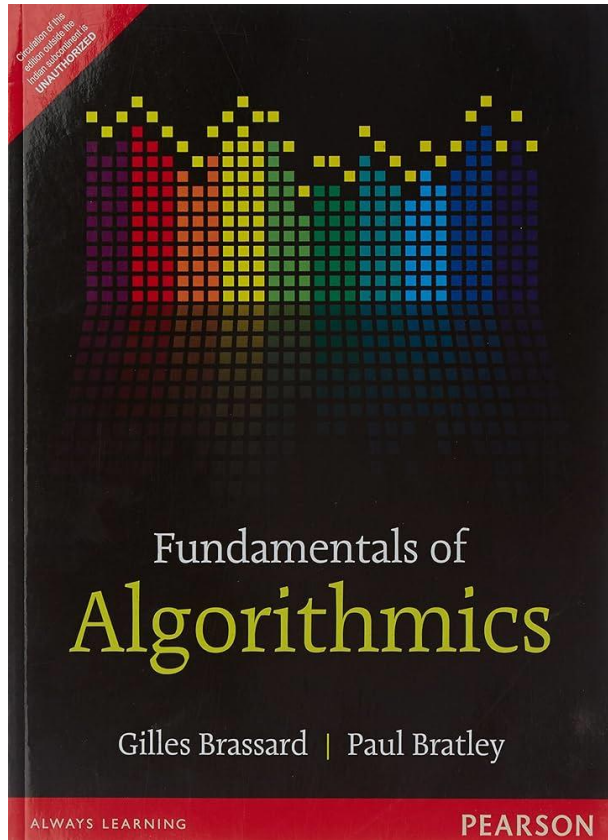


Introduction to Algorithms

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein

Recommended Books

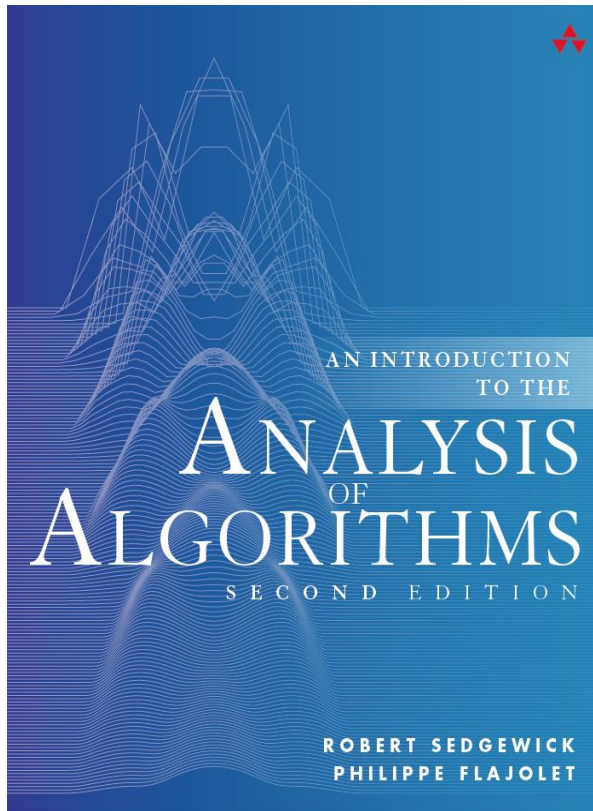
6



Fundamentals of Algorithms Brassard & Bratley

Recommended Books

7



Analysis of Algorithms

Robert Sedgewick

Phillippe Flajolet

ALGORITHM



Algorithm

9

- An algorithm is a well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.
- We can also view an algorithm as a tool for solving a well-specified **computational problem**.
- The algorithm describes a specific computational procedure for achieving input/output relationship.
- Algorithm is finite number of **computational statements** that transform input into output.

Algorithm

10

- An algorithm is a well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.

For Example:

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Algorithm

11

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

For example,

- ▣ given the input sequence (31; 41; 59; 26; 41; 58),
- ▣ a sorting algorithm
- ▣ returns as output sequence (26; 31; 41; 41; 58; 59).
- Such an input sequence is called an ***instance*** of the sorting problem

Algorithm

12

- Finite sequence of instructions.
- Each instruction having a clear meaning.
- Each instruction requires **finite amount of effort**.
- Each instruction requires **finite time to complete**.

PSEUDOCODE

Pseudocode

14

- Pseudocode is a compact and informal high-level description of a program using the conventions of a programming language, but intended more for humans.
- Pseudocode is not an actual programming language.
- So it cannot be compiled into an executable program.

Pseudocode

15

- **INPUT** – indicates a user will be inputting something
- **OUTPUT** – indicates that an output will appear on the screen
- **WHILE** – a loop (iteration that has a condition at the beginning)
- **FOR** – a counting loop (iteration)
- **REPEAT – UNTIL** – a loop (iteration) that has a condition at the end
- **IF – THEN – ELSE** – a decision (selection) in which a choice is made any instructions that occur inside a selection or iteration are usually indented

Pseudocode Example

16

□ Calculate Area and Perimeter of Rectangle

BEGIN

NUMBER b1, b2, area, perimeter

INPUT b1

INPUT b2

area=b1*b2

perimeter=2*(b1+b2)

OUTPUT area

OUTPUT perimeter

END

Pseudocode Example

17

□ If Else Example

```
1  BEGIN
2  NUMBER age
3
4  INPUT "Enter your age for driving licence"
5  OUTPUT age
6
7  IF age>=18 THEN
8      OUTPUT "You can take driving licence"
9  ELSE
10     OUTPUT "You can't take driving licence"
11 ENDIF
12 END
```

ANALYSIS OF ALGORITHM



Algorithm (Correctness)

19

Correct Algorithm:

- An algorithm is said to be correct if, for every **input instance**, it halts with the correct output.

Incorrect algorithm

- An incorrect algorithm,
 - ▣ might not halt at all on some input instances, or
 - ▣ It might halt with an answer other than desired one.

Evaluation of an Algorithm

20

- **Completeness:** is the strategy guaranteed to find a solution when there is one?
- **Optimality:** does the strategy find the highest-quality (least-cost) solution when there are several different solutions?
- **Time complexity:** how long does it take to find a solution?
- **Space complexity:** how much memory is needed to perform the search?

Analysis of an Algorithm

21

- What is the goal of analysis of algorithms?
 - ▣ To compare algorithms mainly **in terms of running time** but also in terms of **other factors** (e.g., memory requirements, programmer's effort etc.)
- What does it mean by **running time analysis**?
 - ▣ **Determine how running time increases as the **size** of the problem increases.**

Analysis of an Algorithm

22

- **Ways of measuring efficiency:**
 - ▣ Run the program and see how long it takes
 - ▣ Run the program and see how much memory it uses

- **Lots of variables to control:**
 - ▣ What is the input data?
 - ▣ What is the hardware platform?
 - ▣ What is the programming language/compiler?

TYPES OF ANALYSIS



Types of Analysis

24

□ Worst case

- ▣ Provides an upper bound on running time
- ▣ An absolute **guarantee** that the **algorithm will not run longer**, no matter what the inputs are.

□ Best case

- ▣ Provides a lower bound on running time
- ▣ Input is the one for which the **algorithm runs the fastest**

Types of Analysis

25

□ Average case

- ▣ Provides a **prediction** about the running time
- ▣ Assumes that the input is random.

$$Lower\ Bound \leq Running\ Time \leq Upper\ Bound$$

Types of Analysis

26

□ Worse Case:

▣ For example, when we are searching for number 23

31	47	15	78	39	17	23
----	----	----	----	----	----	----

□ Best Case:

▣ For example, when we are searching for number 31

31	47	15	78	39	17	23
----	----	----	----	----	----	----

□ Average Case:

▣ For example, when we are searching for any number,

31	47	15	78	39	17	23
----	----	----	----	----	----	----

TIME ANALYSIS



Analysis of an Algorithm

28

- Constant Time
- Logarithmic Time
- Linear Time
- Quadratic Time
- Polynomial Time
- Exponential Time

Analysis of an Algorithm

29

Complexity	Growth Rate	
$O(k) = O(1)$	Constant Time	 Increasing Complexity
$O(\log_b N) = O(\log N)$	Logarithmic Time	
$O(N)$	Linear Time	
$O(N \log N)$		
$O(N^2)$	Quadratic Time	
$O(N^p)$	Polynomial Time	
....		
$O(k^N)$	Exponential Time	

Analysis of a Simple Program

30

```
// Input: int A[N], array of N integers
// Output: Sum of all numbers in array A
int Sum(int A[], int N){
1.   int s=0;
2.   for (int i=0; i< N; i++)
3.       s = s + A[i];
4.   return s;
}
```

How should we analyse this?

Analysis of a Simple Program

31

```
// Input: int A[N], array of N integers
// Output: Sum of all numbers in array A
```

```
int Sum(int A[], int N){
    int s=0; ← ①
```

```
    for (int i=0; i< N; i++) ← ② ③ ④
```

```
        s = s + A[i]; ← ⑤ ⑥ ⑦
```

```
    return s; ← ⑧
}
```

1,2,8: Once

3,4,5,6,7: Once per each iteration
of for loop, N iteration **but**
(3) has one extra iteration

Total: $5N + 3 + 1$

The *complexity function* of the
algorithm is : $f(N) = 5N + 4$

Analysis of a Simple Program

32

- Estimated running time for different values of N for $5N+4$:
 - ▣ $N = 10 \Rightarrow 54$ steps
 - ▣ $N = 100 \Rightarrow 504$ steps
 - ▣ $N = 1,000 \Rightarrow 5004$ steps
 - ▣ $N = 1,000,000 \Rightarrow 5,000,004$ steps
- As N grows, the number of steps grow in linear proportion to N for this function “Sum”

Example 1 Constant time

33

```
void printFElementOfArray(int arr[]) {  
    printf("First element of array = %d",arr[0]);  
}
```

- This function runs in $O(1)$ time (or "constant time") relative to its input.

Constant time statements

34

- Simplest case: $O(1)$ time statements
- Assignment statements of simple data types
 - ▣ `int x = y;`
- Arithmetic operations:
 - ▣ `x = 5 * y + 4 - z;`
- Array referencing:
 - ▣ `A[i] = 5;`
- Array assignment:
 - ▣ `∀ i, A[i] = i;`
- Most conditional tests:
 - ▣ `if (x < 12) ...`

Example 2 Constant time

35

```
int sum = 0, j;  
    for (j = 0; j < 100; j++)  
        sum = sum + j;
```

- Loop executes 100 times
- $4 = O(1)$ steps per iteration
- Total time is $100 * O(1) = O(100 * 1) = O(100) = O(1)$

Example 3 Linear time

36

```
void printAllElementOfArray(int arr[], int size){  
    for (int i = 0; i < size; i++) {  
        printf("%d\n", arr[i]);  
    }  
}
```

- This function runs in $O(n)$ time (or “linear time”) relative to its input, where n is the number of items in the array.
 - ▣ If the array has 10 items, it prints 10 times. If it has 1000 items, it prints 1000 times.

Example 4 Quadratic time

37

```
void printAllPossibleOrderedPairs(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        for (int j = 0; j < size; j++) {  
            printf("%d = %d\n", arr[i], arr[j]);  
        }  
    }  
}
```

- This function runs in $O(n^2)$ time (or “quadratic time”) relative to its input, where n is the number of items in the outer array and n for inner array.
 - If the array has 10 items, it prints 100 times. If it has 1000 items, it prints 1000,000 times.

Example 5 Exponential time

38

```
int fibonacci(int num) {  
    if (num <= 1)  
        return num;  
    return fibonacci(num - 2) + fibonacci(num - 1);  
}
```

- An example of an $O(2^n)$ function is the recursive calculation of Fibonacci numbers.
- The growth curve of an $O(2^n)$ function is exponential.

Example 6 Logarithmic time

39

```
for (int i = 1; i <= n; i *= c)
{
    // some O(1) expressions
}
```

```
for (int i = n; i > 0; i /= c)
{
    // some O(1) expressions
}
```

□ $O(\log n)$

Example 6 Logarithmic time

40

Algorithm that runs in $\log_2(n)$

- ✓ It demonstrates an algorithm that runs in $\log_2(n)$
- ✓ Notice the post operation of the “for loop” multiplies the current value of **i** by 2, so it goes from 1 to 2 to 4 to 8 to 16 to 32 ...

```
for(int i = 1; i <= n; i = i * 2)  
    print "hello";
```

Example 6 Logarithmic time

41

Algorithm that runs in $\log_3(n)$

- ✓ It demonstrates an algorithm that runs in $\log_3(n)$
- ✓ Notice the post operation of the “for loop” multiplies the current value of **i** by 3, so it goes from 1 to 3 to 9 to 27 ...

```
for(int i = 1; i <= n; i = i * 3)  
    print "hello";
```

Example 6 Logarithmic time

42

Algorithm that runs in $\log(n)$

$O(\log n)$ basically means time goes up linearly while the n goes up exponentially.

For Example:

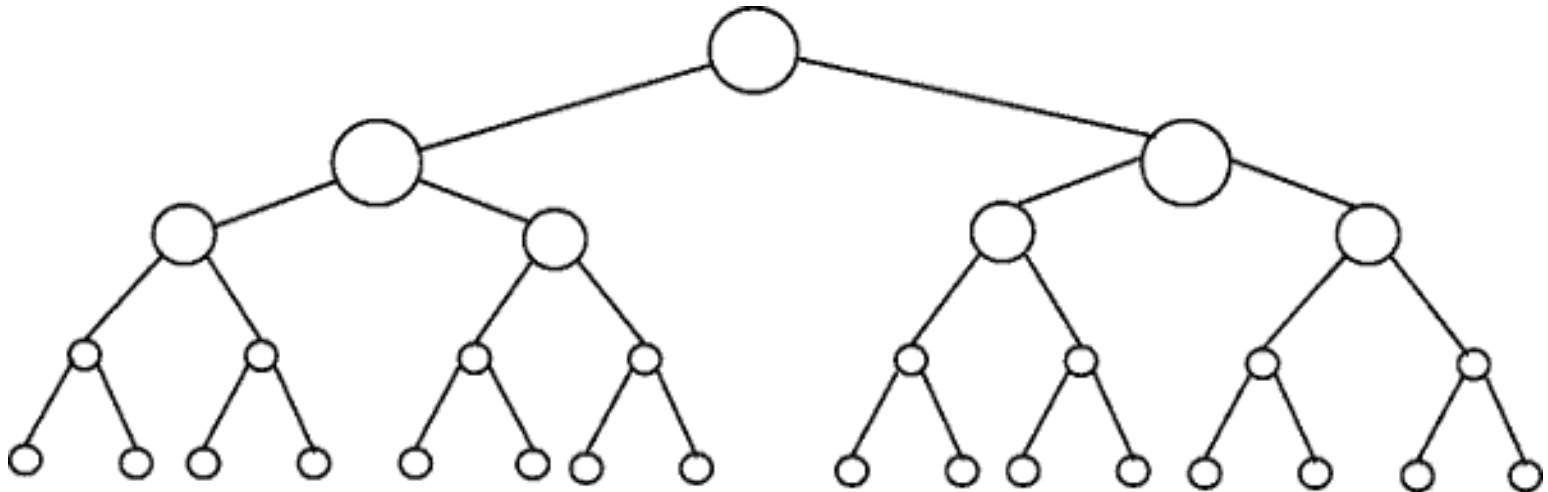
If it takes 1 second to compute 10 elements, it will take 2 seconds to compute 100 elements, 3 seconds to compute 1000 elements, and so on.

Example 6 Logarithmic time

43

Algorithm that runs in $\log(n)$

Binary search is an example with complexity $O(\log n)$

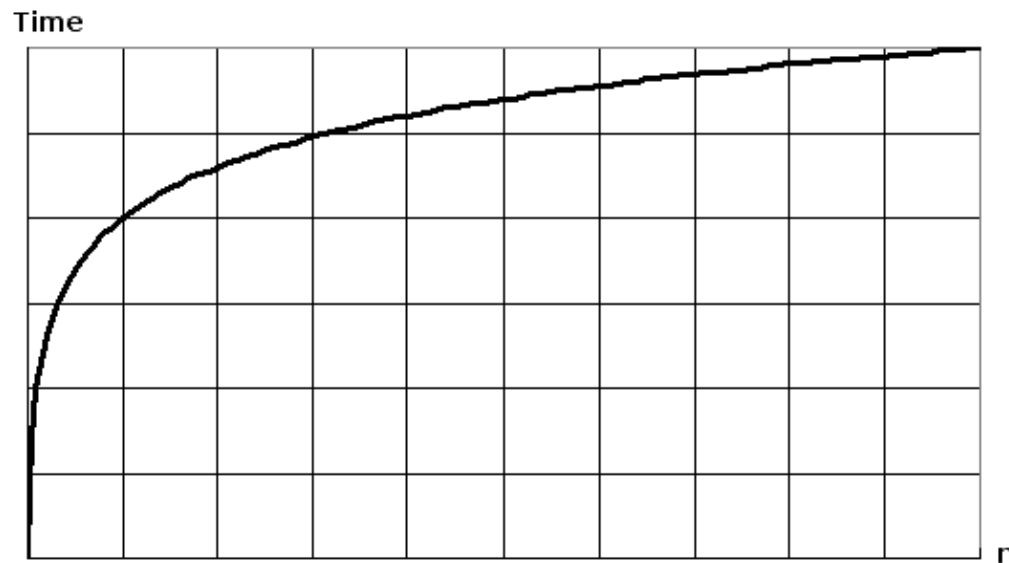


Example 6 Logarithmic time

44

Algorithm that runs in $\log(n)$

$O(\log n)$ basically means time goes up linearly while the n goes up exponentially.



TIME ANALYSIS

A FEW TRICKS



Example 1 “Drop the Constants”

46

```
void printAllItemsTwice(int arr[], int size)
{
    for (int i = 0; i < size; i++) {
        printf("%d\n", arr[i]);
    }
    for (int i = 0; i < size; i++) {
        printf("%d\n", arr[i]);
    }
}
```

- This is $O(2n)$, which we just call $O(n)$.

Example 2 “Drop the Constants”

47

```
void printItem100Times(int arr[], int size) {  
    printf("First element of array = %d\n",arr[0]);  
    for (int i = 0; i < size/2; i++) {  
        printf("%d\n", arr[i]);  
    }  
    for (int i = 0; i < 100; i++) {  
        printf("Hi\n");  
    }  
}
```

□ This is $O(1 + n/2 + 100)$, which we just call $O(n)$.

Example 3 “Drop the less Significant Items”

48

```
void printAllNumbersAndPairSums(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d\n", arr[i]);  
    }  
    for (int i = 0; i < size; i++) {  
        for (int j = 0; j < size; j++) {  
            printf("%d\n", arr[i] + arr[j]);  
        }  
    }  
}
```

- Here our runtime is $O(n + n^2)$, which we just call $O(n^2)$.

Example 4 “Drop the less Significant Items”

49

- Here our runtime is $O(n + n^2)$, which we just call $O(n^2)$.

Similarly,

- $O(n^3 + 160n^2 + 50000)$ is $O(n^3)$
- $O((n + 40) * (n + 4))$ is $O(n^2)$

Example 5

50

```
bool arrayWithElement(int arr[], int size, int element) {  
    for (int i = 0; i < size; i++) {  
        if (arr[i] == element)  
            return true;  
    }  
    return false;  
}
```

- Generally, we say this is $O(n)$ runtime and the "worst case" part would be implied.
- But to be more specific we could say this is **worst case $O(n)$** and the **best-case $O(1)$** runtime.

Comparative Analysis

51

Input Size (n)	$O(1)$	$O(\log(n))$	$O(n)$	$O(n \cdot \log(n))$	$O(n^2)$
1	1	1	1	1	1
4	1	2	4	8	16
16	1	4	16	64	256
1024	1	10	1024	10240	1048576

Acknowledgement

52

Introduction to Algorithms

Thomas H. Cormen, Charles E. Leiserson,
Ronald L. Rivest, and Clifford Stein